Pulsar Signal De-noising with RNN

Isabelle Coughlin and Gabby Tamayo

May 3, 2025

CSC 375: NN and DL Final Project

1 Introduction and Motivation

Puslars are rapidly rotating neutron stars that radiate electromagnetically, and producing a "pulsing" signal aimed at the Earth that we can detect periodically. This "period" can help give us insight into the pulsars behavior, distance, makeup, etc. Yet, standard techniques for cleaning and calculating this period are often complex and inefficient. Therefore, finding a way to quickly decode this signal into the relevant parts is critical to stellar and astrophysics. Moreover, astronomical data is often rife with noise, containing signals from both target objects scattered and miscellaneous signals from surrounding space. Therefore, data cleaning and de-noising are vital pre-processing steps to even begin analysis on most telescopic and astronomical data. Our goal is to use neural network models to de-noise a pulsar signal, transforming it to be smoother and easier to work with for future analysis.

We derived the idea for this project from a paper, [1], titled "Denoising Method of Pulsar Photon Signal Based on Recurrent Neural Network". The paper introduces the theoretical framework for approaching this problem, and shows the results for pulsar x-ray signals from the (RXTE) mission. Since this data was not publicly available, we are testing our results on gamma-ray data from the Large-Array-Telescope, focused in on one pulsar signal. As the paper explains, this problem is unique since there is no "ground-truth" data for the model to learn and compare to. Signal denoising is a great place to implement the use of simulated data, whereby a model is trained exclusively on simulated data and applied onto a target dataset post-training in order to produce results.

A large focus of this project was to explore the differences in multiple networks and the effects of different forms of simulated data on the final results. We trained both a LSTM and Auto-encoder to compare the behavior of the learning and the denoised signals.

2 Data Processing

The data obtained from public release of the Large-Array-Telescope (LAT) had a lot of supplementary information that was not relevant to this exploration. Our focus was on an intensity-vs-time distribution, and in order to obtain that data we had to implement some changes to the original dataset.

The LAT data was organized such that each row corresponded to one observed photon, and tracked the arrival time and energy of each (along with a lot of other features). Since photons are so small and quick in nature, there are millions of samples and the numerical accuracy of the telescope showed that many of them arrived at the same time. Therefore, to have usable data, we binned everything into about 400 time-samples. Hence, the data we are hoping to smooth-out is represented as

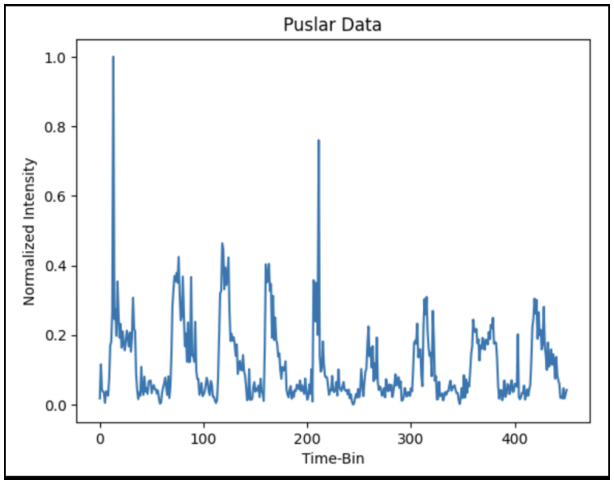

Figure 1: Original Binned Pulsar Data

While this data is not used during training of the model, it is key in understanding the generalizability of our models–trained through simulated data–on more realistic dataset. We also used this graph to understand the behavior of our real data so that we can model a similar behavior in our simulated data. Since our goal is to reproduce a smoother signal specifically for pulsar data of this form, we want to focus the computational power of training to recognize these patterns.

3 Simulated Data

In comparing to 1, we found that our data was best simulated with gaussian pulses. Therefore, for our "clean" data, we created a dataset of normalized signals, each with constant gaussian pulses of random period and pulse width. Furthermore, we experimented with a few techniques for adding noise to the simulated data, but more specifically two different versions that produced interesting and comparable results. We can see examples of these two versions below, what we have labeled Single-Directional Noise and Dual-Directional Noise. The "clean" signal is seen in orange, and is overlayed by the same signal with added noise.

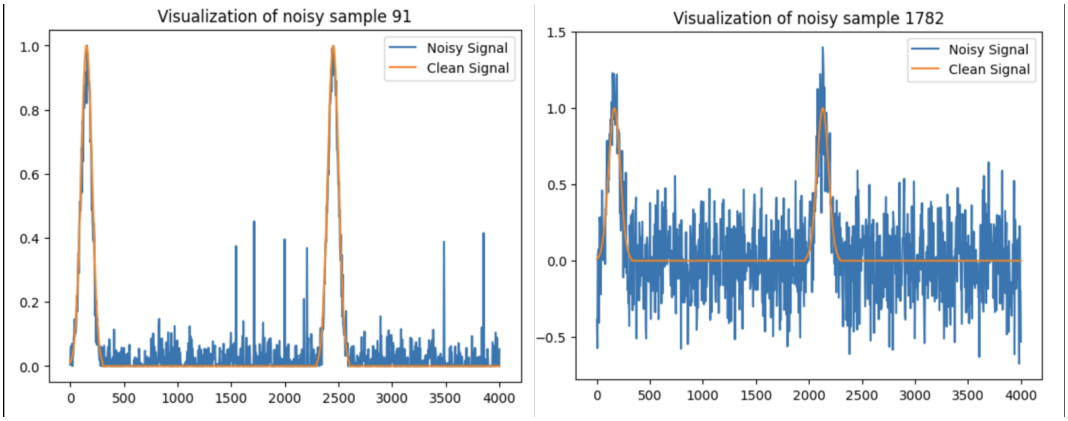

Figure 2: Simulated Noise: Single-Directional (left) and Dual-Directional (right)

On the right, representing the Single-Directional Noise, we have compiled a few noise-adding methods to produce a signal that is quite similar to our real pulse data. It is comprised of adding Additive White Gaussian Noise and Impulse Noise. Each of these methods provide a different noise behavior that the model would need to learn and recognize, which adds added complexity and generalizability, since the data is slightly more realistic and random. The Gaussian Noise, which is pretty standard, adds a random value to the point, within a standard deviation of the data value. Furthermore, the

Impulse Noise contributes the random higher spikes that are seen in the data. We implemented this addition to try and model the much larger spikes seen at the top of a few of the pulses in the original data. A further investigation would be interesting to see the effects of adding just one of these variations of noise, such as only Additive White Gaussian Noise, and analyze the results.

On the left, with the Dual-Directional Noise, we only implement the Additive Gaussian Noise except it is scaled by a random factor that can be positive of negative. Therefore, we see that the entire data is no longer normalized exactly and that there is data below the pulse axis. This type of data is more relatable to other types of signal as well, and therefore may be more generalizable to signals outside of pulsar data. We often see audio, sensor, etc, data in which the noise is much bigger than any of the signal. In Figure 2, our examples show signals with relatively little signal to noise (SNR) ratios, in order to visualize the signal and the noise together and see the relationship, but many of our training examples have much higher SNR than what is seen.

In order to use this model as a predictor for pulsar data, the goal is to make simulated data that is similar to our target data. Our hypothesis was that the Single-Directional Data would more adequately simulate the read data, but we did find conflicting results for each of the models that we tested. A further test could involve making a training dataset that includes examples of both types of noise to make the model even more generalizable.

4 RNN Model

The original paper [1] created an LSTM model, which is well-suited for this project because it is time dependent data and it uses prior information to train more efficiently. While they provided the hyperparameter that they chose, we found that an exact replica of their model would not be very relevant since we have different training datasets with different sizes, complexities, configurations, etc. Therefore, we began by using a single layer LSTM model and continued to experiment with the architecture some. Our final model was organized as seen below in Figure 3.

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 50, 64)	16,896
lstm_1 (LSTM)	(None, 50, 64)	33,024
lstm_2 (LSTM)	(None, 50, 64)	33,024
time_distributed (TimeDistributed)	(None, 50, 1)	65

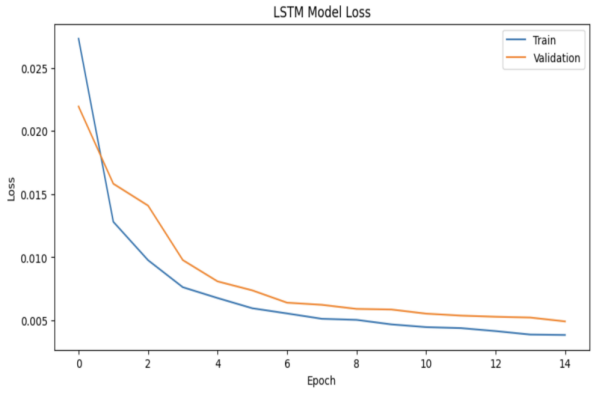

Figure 3: RNN Architecture (left) and Loss Behavior (right)

The LSTM model is well suited for this project because it allows longer-range understanding of the time-dependent behavior that the signal is producing. Especially for signals with oscillations and patterns, the LSTM is apt for finding these and applying them to the entire model. The memory-aspect also allows for better application of prior-learned patterns into the future signals. As we see below in Figure 4, on both datasets, the LSTM model performed quite well in matching the period of oscillation and was not far off from the amplitude of each peak either.

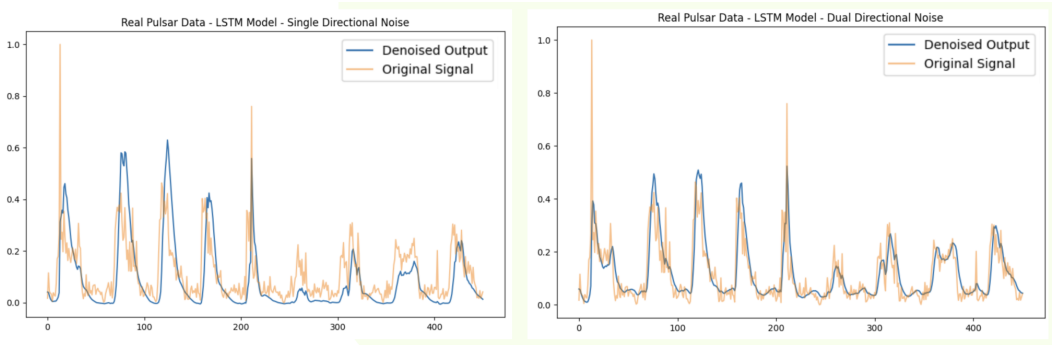

Figure 4: RNN Model Results on Pulsar Data

As we will discuss, there is inherently no way to find an accuracy for the prediction on the real-data, but it is nonetheless useful to visualize and can be used as a tool for continued study. Yet, in Figure 3, on the right, we see the loss curve of the model per epoch based on the validation and training of the simulated data. This is the curve for the Dual-Directional Noise (although the behavior of the curve for both types of noise was similar). We see that the model performs well with a continuously decreasing loss per epoch and shows no signs of overfitting. Our loss function is Mean Squared Error (MSE) since can calculate the distance from the predicted curve to the clean signal provided–common for a regression problem.

5 Autoencoder

Autoencoders are models that learn how to to represent data in a compressed format (encoded) in order to learn the most essential features, and then reconstruct, or decode it, to the original representation. Below, in Figure 5, we see the structure of our autoencoder. The code for this autoencoder was borrowed from the public github repository at [2]. As we can see the first two layers are the encoding section, where it compresses from 128 to 32 channels. Then, it is decoded in the transpose layers, returning from 32 to 128. Lastly, the last layer will augment the output into a single signal dimensions.

Layer (type)	Output Shape	Param #
conv1d (Conv1D)	(None, 148, 128)	512
conv1d_1 (Conv1D)	(None, 146, 32)	12,320
conv1d_transpose (Conv1DTranspose)	(None, 148, 32)	3,104
conv1d_transpose_1 (Conv1DTranspose)	(None, 150, 128)	12,416
conv1d_2 (Conv1D)	(None, 150, 1)	385

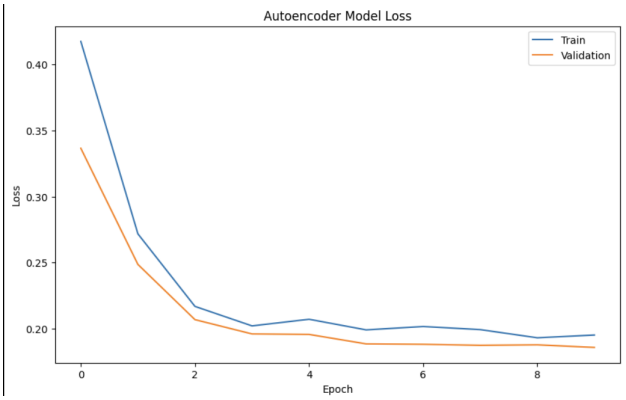

Figure 5: Autoencoder Architecture and Loss Behavior

Since we were attempting to recreate the results from the autoencoder, we did not change any of the architecture or hyper-parameters involved. Nonetheless, the code borrowed from [2] was used to de-noise parabolic curves, so the input of the model did have to be augmented to fit with our problem. The parameters chosen for this model are well-suited for our problem–since signal de-noising is essentially the same question no matter what the curve behavior is. Therefore, we explore the use of the pre-chosen architecture to understand why everything is functioning as it is.

We implement a sigmoid activation function, since the values are normalized to a range between 0 and 1. As we have seen, sigmoid is often used to see how well probability confidence predictions are as compared to a ground-truth prediction. In this model, the sigmoid is being implemented similarly, since it is predicting a value between 0 and 1 and comparing it to the clean signal during training to minimize that distance. Similarly, we implement a binary-cross entropy loss function for evaluating the performance during training. This forces a stronger penalization for smaller differences between the predicted value and the target value–which encourages faster convergence.

As we see on the right in Figure 5, the loss of the model for both training and validation looks like it is behaving generally well, and decreases to become stable without any obvious overfitting. Yet, the loss values are not particularly loss, stabilizing at around 0.2 (as compared to 0.005 for the LSTM model). Looking at the results in Figure 6, we see that the model does not predict a relatively good signal for either training dataset.

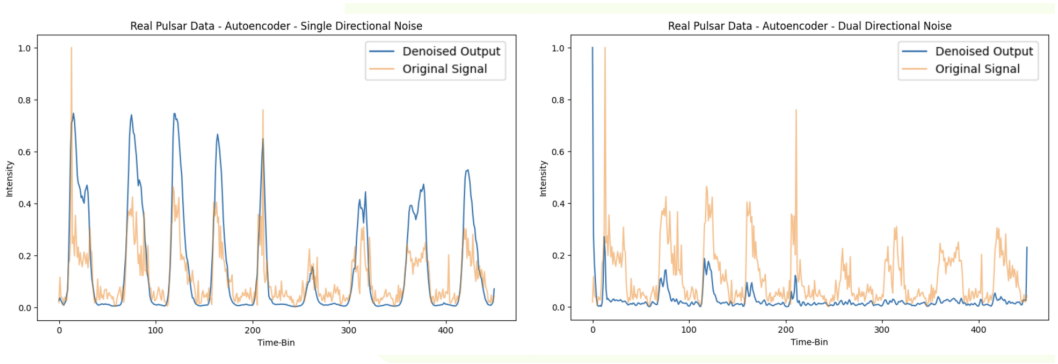

Figure 6: Autoencoder Model Results on Pulsar Data

On the right, we can see that the peaks are definitely more defined and the periodicity looks good, but the amplitude is highly overestimated at each. On the left, the signal severely underestimates each peak–to the extent that some are barely registered. It was particularly interesting to see the difference in the two datasets. While we are still unsure exactly what caused the differing behaviors, it shows the extent that simulated training data can have on the predictions of the model–even if they are different methods of predicting the exact same signals. The amount and type of noise can have an effect of what types of models are more equipped to de-noise them.

In our understanding of autoencoders, we know that they are good for understanding more deeply ingrained patterns–and are better at detecting noise when it is local and not correlated over time. Since the patterns in this dataset are affecting the entire signal and span the entire range, it is harder for the autoencoder to detect them.

6 Comparison

Between the two models, we found that the RNN model performed better on the puslar data than the autoencoder model. This was slightly surprising to use, since we had assumed that the auto-encoder would be the more powerful and sophistocated model to pick up more consistent patterns in the data. Yet, we saw that the LSTM model performed well for both datasets–showing good generalizability and adaptability. Due to the higher memory and ability to get long-range patterns, LSTM was perfect for this task. The autoencoder would require more fine-tuning and specifications to reach a better accuracy and prediction capabilities.

As we mentioned prior, these models are not able to produce an accuracy rating for our real-data, since we do not have access to a ground-truth to compare to. These types of models, once shown reliability through continued testing, could become a tool to clean data more efficiently for astronomical findings. In the specific case of pulsar data, it is necessary to find the period of the data. In order to do this, we must "fold" the data over an undefined number of periods in order for it to

"line-up" and form a repeating pattern. For noisier data, more periods are required for folding—which necessitates more data and computational power. In the original paper, [1], the authors continued with their analysis and compared the number of folds (and the signal-to-noise ratios) required for noisy and de-noised data. The results were substantial and showed that the efficiency of the de-noised signal allows for a drastic simplification in the work required to perform further analysis to the pulsar signals. This is just one example of the numerous applications of this de-noising technique for astrophysics and countless other fields of study.

References

[1] Yu Jiang et al. "Denoising Method of Pulsar Photon Signal Based on Recurrent Neural Network". In: *2019 IEEE International Conference on Unmanned Systems (ICUS)*. 2019, pp. 661–665. DOI: 10.1109/ICUS48101.2019.8996040.

[2] Christian Versloot. *Christianversloot/Keras-autoencoders: Autoencoders and related code, created with Keras.* 2019. URL: https://github.com/christianversloot/keras-autoencoders.